ChatGPT 5.2

Ask ChatGPT

Portfolio Holdings + Purchases System
📄 Business Context (What You Receive at 9:00 AM)
You are building a lightweight Portfolio Management System for internal investment analysts.
Currently, analysts manually track stock purchases and holdings in spreadsheets. The team wants a web application to:
Add securities to a portfolio
Record buy/sell transactions
Automatically compute holdings
View portfolio summary
Your goal is to build a working full-stack prototype in ~4.5 hours.
You may use AI tools.
🎯 Functional Requirements
1️⃣ Portfolio Holdings
Each holding should contain:
Symbol (e.g., AAPL)
Company Name
Quantity Owned
Average Cost
Total Invested
Current Market Value (assume price is manually entered for now)
The system must:
Display all holdings
Update holdings automatically when transactions are recorded
Prevent negative holdings
2️⃣ Transactions
Users should be able to:
Record BUY transaction:
symbol
quantity
price
timestamp
Record SELL transaction:
symbol
quantity
price
timestamp
Rules:
Cannot sell more than owned quantity
Average cost should update correctly for BUY
SELL should reduce quantity but not change avg cost logic incorrectly
3️⃣ Portfolio Summary View
Display:
Total invested amount
Total market value
Unrealized gain/loss
Gain/Loss %

help me build the backend system with python and fastapi, SQL, lets go step by step,
1. installation and setup
2. Architecture
3. Database schema

4. Functional requirements + Database data population

Option A (fast prototype, very clear)
Store transactions
Maintain a holdings table that is updated whenever a transaction is recorded
Multi User

lets go step by step,

this is for Hack2hire interview event at deloitte

4/4

Alright—let's build **Option A** (store transactions + maintain a holdings table updated on each trade), **multi-user**, with **FastAPI + SQLAlchemy + Postgres**. I'll keep it hackathon-friendly but "interview-clean".

Below is **Step 1: Installation & Setup** (you can copy/paste). After this, we'll do Step 2 (Architecture) and start writing code.

Step 1 — Installation & Setup

1) Create project folder + venv

Bash

```bash
mkdir portfolio-pms-backend
cd portfolio-pms-backend

python -m venv .venv
# mac/linux:
source .venv/bin/activate
# windows (powershell):
# .\.venv\Scripts\Activate.ps1
```

2) Install dependencies

For a prototype that's still production-ish:

Bash

```bash
pip install fastapi uvicorn[standard] sqlalchemy alembic psycopg2-binary pydantic-sett
```

Optional but useful:

Bash

```bash
pip install passlib[bcrypt] python-jose[cryptography]
```

(Those are for password hashing + JWT auth; since you said multi-user, we'll likely want them.)

3) Create a minimal folder structure

Bash

```
mkdir -p app/{api,core,db,models,schemas,services}
touch app/main.py app/__init__.py
touch app/core/config.py
touch app/db/session.py app/db/base.py
touch .env
```

Recommended structure (we'll fill this in step 2):

Code

```
app/
  main.py
  api/            # routers (endpoints)
  core/           # settings, security
  db/             # engine/session, base
  models/         # SQLAlchemy models
  schemas/        # Pydantic schemas
  services/       # business logic (holdings updates, validations)
alembic/          # migrations
.env
```

4) Choose database

Postgres is ideal for Deloitte-style interviews. Easiest local setup is Docker.

Option 1: Postgres via Docker (recommended)

Create `docker-compose.yml` :

Bash

```
touch docker-compose.yml
```

Paste:

YAML

```yaml
version: "3.9"
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: portfolio
      POSTGRES_PASSWORD: portfolio
      POSTGRES_DB: portfolio_db
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Start it:

Bash

```
docker compose up -d
```

Option 2: SQLite (fastest)

If you want maximum speed and no Docker: use SQLite.

But since you asked "SQL + multi-user", I'd still go Postgres.

5) Configure environment variables

Edit `.env`:

```env
DATABASE_URL=postgresql+psycopg2://portfolio:portfolio@localhost:5432/portfolio_db
APP_NAME=Portfolio PMS
ENV=dev
```

6) Add config + DB session code

Create `app/core/config.py`:

```python
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    DATABASE_URL: str
    APP_NAME: str = "Portfolio PMS"
    ENV: str = "dev"

settings = Settings()
```

Create `app/db/session.py`:

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from app.core.config import settings

engine = create_engine(settings.DATABASE_URL, pool_pre_ping=True)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Create `app/db/base.py`:

```python
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass
```

Create `app/main.py`:

```python
```

```python
from fastapi import FastAPI
from app.core.config import settings

app = FastAPI(title=settings.APP_NAME)

@app.get("/health")
def health():
    return {"status": "ok", "env": settings.ENV}
```

Run server:

```bash
uvicorn app.main:app --reload
```

Check:

- http://127.0.0.1:8000/health
- http://127.0.0.1:8000/docs

7) Set up Alembic migrations

Initialize:

```bash
alembic init alembic
```

Now edit `alembic.ini` and set:

```
sqlalchemy.url = postgresql+psycopg2://portfolio:portfolio@localhost:5432/portfolio_db
```

Then edit `alembic/env.py` so Alembic knows your Base metadata. Find `target_metadata = None` and change:

```python
from app.db.base import Base
target_metadata = Base.metadata
```

Also ensure `app` is importable (normally works if you run commands from repo root with venv active).

What “done” looks like after Step 1

You should have:

- FastAPI running (`/docs` works)
- Postgres running (if Docker)
- Alembic initialized and pointed at your DB

Next: Step 2 — Architecture (what we’ll build)

We’ll implement:

- multi-user portfolios (User → Portfolio → Transactions + Holdings)
- transaction write endpoint that:

  - validates sell rules
  - writes transaction
  - atomically updates holdings row (quantity, avg_cost, invested, market_value)
- portfolio summary endpoint

- Define modules/routers/services pattern
- Define “transaction posting flow” (single DB transaction)
- Decide whether we allow m

Say “go Step 2” and I’ll lay out t

I want to do postgres, but without docker, I have already Postgres installed in my system. its already connected through dbvisualizer

Perfect—if Postgres is already installed and you can see it in DbVisualizer, you just need:

1. a database + user (or reuse what you have)
2. the correct **SQLAlchemy connection string**
3. make sure FastAPI can connect

Here’s Step 1 adjusted for **local Postgres (no Docker)**.

---

1) Create a database + user (if you don’t already have one)

Option A: Use psql (recommended)

Open a terminal:

```bash
psql -U postgres
```

Then run:

```sql
CREATE USER portfolio WITH PASSWORD 'portfolio';
CREATE DATABASE portfolio_db OWNER portfolio;
GRANT ALL PRIVILEGES ON DATABASE portfolio_db TO portfolio;
```

Exit:

```sql
\q
```

Option B: Use DbVisualizer

You can also create the DB and user in DbVisualizer (SQL Commander). Same SQL as above.

---

2) Confirm host/port

Most local installs are:

- host: `localhost`
- port: `5432`

If your DbVisualizer connection uses a different port, use that.

Quick check from terminal:

```bash
psql -h localhost -p 5432 -U portfolio -d portfolio_db
```

If that works, you're good.

---

3) Update `.env` connection string (SQLAlchemy format)

In your `.env`:

```env
DATABASE_URL=postgresql+psycopg2://portfolio:portfolio@localhost:5432/portfolio_db
APP_NAME=Portfolio PMS
ENV=dev
```

If your password has special characters, URL-encode it.

---

4) Make sure Python dependencies are correct

In your venv:

```bash
pip install psycopg2-binary sqlalchemy alembic fastapi uvicorn[standard] pydantic-sett
```

---

5) Quick connectivity test (before we build more)

Create `test_db.py` in project root:

```python
from sqlalchemy import create_engine, text
from app.core.config import settings

engine = create_engine(settings.DATABASE_URL, pool_pre_ping=True)

with engine.connect() as conn:
    print(conn.execute(text("select version()")).one())
```

Run:

```bash
python test_db.py
```

You should see a Postgres version string.

---

6) Alembic config (local Postgres)

In `alembic.ini`, set:

Code

```
sqlalchemy.url = postgresql+psycopg2://portfolio:portfolio@localhost:5432/portfolio_db
```

And in `alembic/env.py` keep:

Python

```python
from app.db.base import Base
target_metadata = Base.metadata
```

Common local Postgres gotchas

- **Password auth**: If `psql` works but Python doesn't, check `pg_hba.conf` (sometimes local is set to `peer` auth). Easiest fix is to connect via TCP (`localhost`) and ensure md5/scram for local connections.
- **Different port**: match what DbVisualizer uses.
- **Multiple Postgres installs**: ensure the service you're connecting to is the same one Python hits.

Next: Step 2 — Architecture

Now that local Postgres is set, we'll define:

- modules/routers/services
- multi-user model
- transaction → holdings update workflow (atomic DB transaction)

To avoid any back-and-forth: **assume**

- one portfolio per user for the hackathon (we can extend later)
- holdings updated immediately on each transaction

If you tell me **your DbVisualizer connection details** (host, port, db name, username), I'll mirror them exactly in the schema + migrations. Otherwise, we proceed with `portfolio_db / portfolio / localhost:5432` .

lets move to step 2

Step 2 — Architecture (FastAPI + SQLAlchemy, Option A, Multi-User)

We'll structure this so you can ship fast, but it still reads like a clean backend.

2.1 High-level components

Core entities

- **User**: who owns portfolios/holdings/transactions

- **Holding**: current position per user per symbol (cached/maintained table)
- **Transaction**: immutable ledger of BUY/SELL events
- *(Optional later)* **Security**: symbol → company name, etc. (can be in holdings for prototype)

Key design choice (Option A)

- **Transactions are the source of truth for audit**
- **Holdings is a derived/cached table** updated whenever a transaction is recorded
  (fast reads for "portfolio view" + easy summary)

Invariants we enforce

- No negative holdings (can't sell more than owned)
- BUY updates avg cost correctly
- SELL reduces quantity and total invested proportionally, **avg_cost stays the same**
- All updates happen inside **one DB transaction** to avoid race bugs

---

2.2 Folder structure (what goes where)

Code

```
app/
  main.py

  api/
    deps.py                # get_db, get_current_user (later)
    routers/
      auth.py              # optional for hack (JWT)
      transactions.py      # POST buy/sell
      holdings.py          # GET holdings
      summary.py           # GET portfolio summary

  core/
    config.py              # env config
    security.py            # password hashing + JWT (optional)

  db/
    base.py
    session.py

  models/
    user.py
    holding.py
    transaction.py

  schemas/
    user.py
    transaction.py
    holding.py
    summary.py

  services/
    portfolio_service.py   # business logic: apply_transaction(), compute_summary()
```

Rule of thumb

- `api/routers` should be thin: parse request → call service → return response
- `services/` holds the business rules (sell validation, avg cost math)
- `models/` is database-only
- `schemas/` is API input/output

2.3 Request/flow design (what endpoints do)

Transactions

- `POST /transactions`
  - body: `{ type: "BUY"|"SELL", symbol, quantity, price, timestamp? }`
  - does:
    1. validate quantity/price > 0
    2. lock the holding row (or create it) **FOR UPDATE**
    3. apply business rules
    4. insert transaction row
    5. update holdings row
    6. commit

Holdings

- `GET /holdings`
  - returns rows from holdings table (fast)

Summary

- `GET /summary`
  - aggregates holdings: total invested, total market value, unrealized, %

2.4 Concurrency & correctness (important for “multi user”)

Why we need row locks

Two sells at the same time could both pass validation and push quantity negative unless we lock.

We’ll do:

- Fetch the holding row using `SELECT ... FOR UPDATE` (SQLAlchemy: `with_for_update()`)
- Update it inside the same DB transaction

That gives correct behavior per `(user_id, symbol)`.

2.5 Holdings math rules (precise)

Let:

- `qty` = quantity owned
- `avg_cost` = average cost per share
- `total_invested` = cost basis (not cash flow)
- `market_price` = manually entered (for now)
- `market_value = qty * market_price`

BUY(q, p)

- new_qty = qty + q
- new_total_invested = total_invested + (q * p)
- new_avg_cost = new_total_invested / new_qty

SELL(q, p)

- must have `qty >= q`
- new_qty = qty - q

- reduce invested by sold shares * avg_cost (cost basis)
  - new_total_invested = total_invested - (q * avg_cost)
- avg_cost stays the same *unless qty becomes 0*
  - if new_qty == 0: set avg_cost = 0 and total_invested = 0

Unrealized P/L:

- `(market_price - avg_cost) * qty`

  or
- `market_value - total_invested`

2.6 Multi-user approach (fast & interview-friendly)

Option 1 (fastest): single "demo user"

- Create one user row at startup / seed
- Hardcode `user_id=1` in endpoints
- Still "multi-user capable" via schema
- Great for hack2hire if time is tight

Option 2 (still fast): JWT auth

- `/auth/register`, `/auth/login`
- Token identifies user
- Endpoints use `get_current_user()`

Recommendation for 4.5 hours: start with Option 1, then add JWT if time remains.

2.7 What we'll implement next (Step 3)

In Step 3 we'll define the **DB schema** with:

- users
- holdings (unique: user_id + symbol)
- transactions (FK to user, index on user_id + symbol + timestamp)

Then we'll write migrations with Alembic.

Quick action items for you (so we don't get stuck)

Before Step 3, decide:

1. Do you want **JWT auth** now, or **demo user** first?
   - If unsure: we'll do demo user first and keep the design ready for JWT.
2. Any need for **multiple portfolios per user**?
   - For hackathon: assume **one portfolio per user**.

If you say nothing, I'll proceed with:

- **one portfolio per user (implicit)**
- **demo user seeding**
- schema ready to add JWT later

Say "go Step 3" and I'll give you the exact SQLAlchemy models + Alembic migration setup and commands.

no auth

Got it — **no auth**. We'll still keep it "multi-user capable" by requiring a `user_id` in requests (or defaulting to 1), but we won't build login/JWT.

Updated Step 2 Architecture (No Auth)

How we identify the user

- **Simplest**: every endpoint accepts `user_id` (query param or in body)
- For hack speed, we'll do:
  - `POST /transactions` body includes `user_id`
  - `GET /holdings?user_id=...`
  - `GET /summary?user_id=...`

(If you want, we can also default `user_id=1` when omitted.)

Service flow (core of correctness)

`POST /transactions`

- DB transaction (atomic)
- Steps:
  1. Validate input (qty > 0, price > 0, type in BUY/SELL)
  2. Load holding row for `(user_id, symbol)` **FOR UPDATE**
     - if missing and BUY: create with qty=0
     - if missing and SELL: reject
  3. Apply BUY/SELL rules:
     - SELL cannot exceed owned qty
     - BUY updates avg_cost + invested
     - SELL reduces qty and invested by `sold_qty * avg_cost`; avg_cost unchanged unless qty

hits 0

4. Insert immutable transaction row
5. Update holdings row
6. Commit, return updated holding + transaction

This is the heart of the app and what interviewers care about.

Minimal endpoints

Transactions

- `POST /transactions`
- request:

```JSON
{
  "user_id": 1,
  "type": "BUY",
  "symbol": "AAPL",
  "company_name": "Apple Inc",
  "quantity": 10,
  "price": 180.5,
  "timestamp": "2026-02-27T09:00:00Z"
}
```

Notes:

- `company_name` optional but handy for holdings display
- `timestamp` optional (default server time)

Holdings

- `GET /holdings?user_id=1`
- shows: symbol, company_name, qty, avg_cost, total_invested, market_price, market_value

Summary

- `GET /summary?user_id=1`
- returns totals + gain/loss %

Next step

Step 3 — Database schema

We'll create tables:

- `users` (just id + name for now; no auth fields)
- `holdings` (unique on user_id + symbol)
- `transactions` (ledger)

And we'll add indexes + constraints.

Say **"go Step 3"** and I'll provide:

- SQLAlchemy models (User, Holding, Transaction)
- Alembic migration commands + initial migration code
- Notes on constraints (prevent negative holdings at app layer + optional DB constraints)

go to step 3

Step 3 — Database Schema (Postgres) + SQLAlchemy Models + Alembic

We'll create 3 tables:

1) `users`

Minimal (no auth):

- `id` (PK)
- `name` (optional)
- `created_at`

2) `holdings` (cached current positions)

One row per `(user_id, symbol)`:

- `symbol`
- `company_name`
- `quantity` (>= 0)
- `avg_cost` (>= 0)
- `total_invested` (>= 0) ← cost basis
- `market_price` (>= 0, manually entered for now)
- timestamps

  Constraints:
- `UNIQUE(user_id, symbol)`
- `CHECK(quantity >= 0)`, etc.

3) `transactions` (immutable ledger)

- `type` BUY/SELL
- `symbol`
- `quantity` (> 0)
- `price` (> 0)
- `timestamp`
- `created_at`

3.1 Create the SQLAlchemy models

A) Update `app/db/base.py` (important for Alembic autogenerate)

Replace your `app/db/base.py` with:

Python

```python
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass

# Import models so Alembic "sees" them
from app.models.user import User  # noqa: E402,F401
from app.models.holding import Holding  # noqa: E402,F401
from app.models.transaction import Transaction  # noqa: E402,F401
```

B) Create `app/models/user.py`

Python

```python
from sqlalchemy import DateTime, Integer, String, func
from sqlalchemy.orm import Mapped, mapped_column

from app.db.base import Base

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str | None] = mapped_column(String(200), nullable=True)

    created_at: Mapped[object] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), nullable=False
    )
```

C) Create `app/models/holding.py`

Use `Numeric` for money-like values.

Python

```python
from sqlalchemy import (
    DateTime,
    ForeignKey,
    Index,
    Integer,
    Numeric,
    String,
    UniqueConstraint,
    CheckConstraint,
    func,
)
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base

class Holding(Base):
    __tablename__ = "holdings"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)

    user_id: Mapped[int] = mapped_column(
        Integer, ForeignKey("users.id", ondelete="CASCADE"), nullable=False
    )

    symbol: Mapped[str] = mapped_column(String(12), nullable=False)
    company_name: Mapped[str | None] = mapped_column(String(255), nullable=True)

    quantity: Mapped[int] = mapped_column(Integer, nullable=False, default=0)

    # cost basis tracking
    avg_cost: Mapped[float] = mapped_column(Numeric(18, 6), nullable=False, default=0)
    total_invested: Mapped[float] = mapped_column(Numeric(18, 2), nullable=False, defa

    # manually maintained for now
    market_price: Mapped[float] = mapped_column(Numeric(18, 6), nullable=False, defaul

    created_at: Mapped[object] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), nullable=False
    )
    updated_at: Mapped[object] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        onupdate=func.now(),
        nullable=False,
    )

    user = relationship("User", lazy="joined")

    __table_args__ = (
        UniqueConstraint("user_id", "symbol", name="uq_holdings_user_symbol"),
        CheckConstraint("quantity >= 0", name="ck_holdings_quantity_nonneg"),
        CheckConstraint("avg_cost >= 0", name="ck_holdings_avg_cost_nonneg"),
        CheckConstraint("total_invested >= 0", name="ck_holdings_total_invested_nonneg
        CheckConstraint("market_price >= 0", name="ck_holdings_market_price_nonneg"),
        Index("ix_holdings_user_symbol", "user_id", "symbol"),
    )
```

Note: I used `int` for quantity; if you want fractional shares, change to `Numeric(18,6)`.

D) Create `app/models/transaction.py`

Python

```python
import enum

from sqlalchemy import (
    DateTime,
    Enum,
    ForeignKey,
    Index,
    Integer,
    Numeric,
    String,
    CheckConstraint,
    func,
)
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base

class TransactionType(str, enum.Enum):
    BUY = "BUY"
    SELL = "SELL"

class Transaction(Base):
    __tablename__ = "transactions"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)

    user_id: Mapped[int] = mapped_column(
        Integer, ForeignKey("users.id", ondelete="CASCADE"), nullable=False
    )

    type: Mapped[TransactionType] = mapped_column(
        Enum(TransactionType, name="transaction_type"),
        nullable=False
    )

    symbol: Mapped[str] = mapped_column(String(12), nullable=False)
    company_name: Mapped[str | None] = mapped_column(String(255), nullable=True)

    quantity: Mapped[int] = mapped_column(Integer, nullable=False)
    price: Mapped[float] = mapped_column(Numeric(18, 6), nullable=False)

    # When trade happened
    timestamp: Mapped[object] = mapped_column(
        DateTime(timezone=True), nullable=False, server_default=func.now()
    )

    created_at: Mapped[object] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), nullable=False
    )

    user = relationship("User", lazy="joined")

    __table_args__ = (
        CheckConstraint("quantity > 0", name="ck_transactions_quantity_pos"),
        CheckConstraint("price > 0", name="ck_transactions_price_pos"),
        Index("ix_transactions_user_ts", "user_id", "timestamp"),
        Index("ix_transactions_user_symbol_ts", "user_id", "symbol", "timestamp"),
    )
```

3.2 Alembic: generate and apply migration

1) Ensure Alembic is configured

In `alembic/env.py` you should have:

```python
from app.db.base import Base
target_metadata = Base.metadata
```

2) Create the migration

```bash
alembic revision --autogenerate -m "init schema"
```

3) Apply it

```bash
alembic upgrade head
```

4) Confirm tables exist (via DbVisualizer or psql)

You should see:

- `users`
- plus the enum type `transaction_type` in Postgres

3.3 Prototype-friendl

Since no auth, we'll seed a coup

```sql
INSERT INTO users (name) VALUES ('Analyst 1'), ('Analyst 2');
```

Short answer:

👉 **You should NOT manually create tables with SQL**
👉 **SQLAlchemy models do NOT automatically create tables unless you tell them to**
👉 Since you're using **Alembic**, the tables are created when you run migrations**

The Correct Way (Professional / Deloitte-ready)

Because you initialized **Alembic**, the flow is:

1️⃣ **You define models (User, Holding, Transaction)**

2️⃣ **Run:**

```bash
alembic revision --autogenerate -m "init schema"
```

This:

- Compares your models
- Generates SQL migration file

do I need to create the table using sql query manually or models sqlalchemy orm will do it??

if ORM will create, when will it create it ?

Next: Step 4 — Functional requirements + data population

We'll implement:

- `POST /transactions` (BUY/SELL) with correct holding updates + `FOR UPDATE` lock
- `GET /holdings`
- `GET /portfolio`
- seed script / startup seed

Say **"go Step 4"** and I'll give you the actual FastAPI routers + the service function `apply_transaction()` (the most important part).

3️⃣ Then run:

```bash
alembic upgrade head
```

🚀 **THIS is when the tables are actually created in Postgres.**

What NOT to do

Do NOT add this in your app:

```python
Base.metadata.create_all(bind=engine)
```

That is:

- Fine for quick prototypes
- ❌ Not good when using Alembic
- ❌ Can cause schema drift
- ❌ Interviewers don't like mixing both approaches

Two Ways Tables Can Be Created

Option A — Using Alembic (Correct Way)

Tables created when you run:

```bash
alembic upgrade head
```

This executes generated SQL against Postgres.

Option B — Using SQLAlchemy Directly (Not recommended here)

If you add somewhere:

```python
from app.db.base import Base
from app.db.session import engine

Base.metadata.create_all(bind=engine)
```

Then tables are created:

- When that line executes
- Usually at app startup

But since you're using Alembic, don't mix.

How To Verify Tables Were Created

After:

```bash
alembic upgrade head
```

Open DbVisualizer and run:

```sql
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public';
```

You should see:

- users
- holdings
- transactions
- alembic_version

Quick Interview-Ready Explanation

If asked:

> "How are your tables created?"

You say:

> “I’m using SQLAlchemy models with Alembic for schema migrations.
> Models define the schema, Alembic autogenerates migration scripts, and tables are created when running `alembic upgrade head`.
> This allows version-controlled, production-safe schema evolution.”

That answer = strong signal.

Important: If Autogenerate Creates Empty Migration

If `alembic revision --autogenerate` creates an empty migration, it means:

- Your models were not imported in `Base.metadata`
- `target_metadata` not set correctly in `env.py`
- You ran command outside project root

Make sure in:

`app/db/base.py`

```python
from app.models.user import User
from app.models.holding import Holding
from app.models.transaction import Transaction
```

Otherwise Alembic doesn’t “see” your tables.

Where You Are Now

Since you already created models, next step should be:

```bash
alembic revision --autogenerate -m "init schema"
alembic upgrade head
```

Then confirm tables exist.

When that works, say:

👉 **"migrations done"**

Then we'll implement the real logic — the transaction service with row locking (the part that wins interviews).

Step 4 — Functional requirements + data population

Step 4 — Functional requirements + DB data population (FastAPI + SQLAlchemy, Option A)

We'll implement:

1. **POST /transactions** (BUY/SELL)
2. **GET /holdings**
3. **PATCH /holdings/{symbol}/market-price** (manual price input)
4. **GET /summary**
5. **Seed users** (simple script + optional endpoint)

All correctness (no negative holdings, avg cost rules, SELL rules) will live in a single service: `apply_transaction()`.

4.1 Pydantic Schemas

Create `app/schemas/transaction.py`

```python
```

```python
from datetime import datetime
from decimal import Decimal
from enum import Enum
from pydantic import BaseModel, Field

class TransactionType(str, Enum):
    BUY = "BUY"
    SELL = "SELL"

class TransactionCreate(BaseModel):
    user_id: int = Field(..., ge=1)
    type: TransactionType
    symbol: str = Field(..., min_length=1, max_length=12)
    company_name: str | None = Field(default=None, max_length=255)
    quantity: int = Field(..., gt=0)
    price: Decimal = Field(..., gt=0)
    timestamp: datetime | None = None  # if None, server time will be used

class TransactionOut(BaseModel):
    id: int
    user_id: int
    type: TransactionType
    symbol: str
    company_name: str | None
    quantity: int
    price: Decimal
    timestamp: datetime
    created_at: datetime

    class Config:
        from_attributes = True
```

Create `app/schemas/holding.py`

Python

```python
from datetime import datetime
from decimal import Decimal
from pydantic import BaseModel, Field

class HoldingOut(BaseModel):
    user_id: int
    symbol: str
    company_name: str | None
    quantity: int
    avg_cost: Decimal
    total_invested: Decimal
    market_price: Decimal
    market_value: Decimal
    created_at: datetime
    updated_at: datetime

    class Config:
        from_attributes = True

class MarketPriceUpdate(BaseModel):
    market_price: Decimal = Field(..., ge=0)
```

Create `app/schemas/summary.py`

Python

```python
from decimal import Decimal
from pydantic import BaseModel

class PortfolioSummary(BaseModel):
    user_id: int
    total_invested: Decimal
    total_market_value: Decimal
    unrealized_gain_loss: Decimal
    gain_loss_pct: Decimal | None  # None if invested is 0
```

4.2 API deps

Create `app/api/deps.py`

Python

```python
from app.db.session import get_db

__all__ = ["get_db"]
```

4.3 Portfolio service (the important part)

Create `app/services/portfolio_service.py`

Python

```python
from __future__ import annotations

from datetime import datetime, timezone
from decimal import Decimal, ROUND_HALF_UP

from fastapi import HTTPException
from sqlalchemy.orm import Session

from app.models.holding import Holding
from app.models.transaction import Transaction, TransactionType
from app.models.user import User

MONEY_2DP = Decimal("0.01")
PRICE_6DP = Decimal("0.000001")

def _d(x) -> Decimal:
    # Ensure Decimal math
    return x if isinstance(x, Decimal) else Decimal(str(x))

def _round_money(x: Decimal) -> Decimal:
    return x.quantize(MONEY_2DP, rounding=ROUND_HALF_UP)

def _round_price(x: Decimal) -> Decimal:
    return x.quantize(PRICE_6DP, rounding=ROUND_HALF_UP)

def ensure_user_exists(db: Session, user_id: int) -> None:
    exists = db.query(User.id).filter(User.id == user_id).first()
    if not exists:
        raise HTTPException(status_code=404, detail=f"user_id={user_id} not found")

def get_holding_for_update(db: Session, user_id: int, symbol: str) -> Holding | None:
    return (
        db.query(Holding)
        .filter(Holding.user_id == user_id, Holding.symbol == symbol)
        .with_for_update()
        .one_or_none()
```

```
    )

def apply_transaction(
    db: Session,
    *,
    user_id: int,
    type: TransactionType,
    symbol: str,
    company_name: str | None,
    quantity: int,
    price: Decimal,
    timestamp: datetime | None,
) -> tuple[Transaction, Holding]:
    """
    Atomic: insert transaction + update holdings with row lock.
    Rules:
      - Cannot sell more than owned
      - BUY updates avg_cost and total_invested
      - SELL reduces quantity and reduces total_invested by qty_sold * avg_cost
        (avg_cost unchanged unless quantity hits 0)
    """
    ensure_user_exists(db, user_id)

    symbol = symbol.upper().strip()
    price = _round_price(_d(price))

    if timestamp is None:
        timestamp = datetime.now(timezone.utc)
    elif timestamp.tzinfo is None:
        # treat naive datetimes as UTC for simplicity
        timestamp = timestamp.replace(tzinfo=timezone.utc)

    # Lock holding row for this user+symbol to prevent race conditions
    holding = get_holding_for_update(db, user_id, symbol)

    if holding is None:
        if type == TransactionType.SELL:
            raise HTTPException(status_code=400, detail="Cannot sell a symbol with no
        holding = Holding(
            user_id=user_id,
            symbol=symbol,
            company_name=company_name,
            quantity=0,
            avg_cost=Decimal("0"),
            total_invested=Decimal("0"),
            market_price=Decimal("0"),
        )
        db.add(holding)
        db.flush()  # assign holding.id / persist before calculations

    # Keep company_name updated if provided
    if company_name:
        holding.company_name = company_name

    qty = int(holding.quantity)
    avg_cost = _d(holding.avg_cost)
    invested = _d(holding.total_invested)

    q = int(quantity)

    if type == TransactionType.BUY:
        # new totals
        buy_cost = _d(q) * price
        new_qty = qty + q
        new_invested = invested + buy_cost

        # avg cost = total_invested / total_shares
        new_avg_cost = (new_invested / _d(new_qty)) if new_qty > 0 else Decimal("0")
```

```
        holding.quantity = new_qty
        holding.total_invested = _round_money(new_invested)
        holding.avg_cost = _round_price(new_avg_cost)

    elif type == TransactionType.SELL:
        if q > qty:
            raise HTTPException(status_code=400, detail=f"Cannot sell {q}; only {qty} 

        new_qty = qty - q

        # Reduce cost basis by shares sold * avg_cost (NOT sale proceeds)
        reduction = _d(q) * avg_cost
        new_invested = invested - reduction

        if new_qty == 0:
            holding.quantity = 0
            holding.total_invested = Decimal("0.00")
            holding.avg_cost = Decimal("0.000000")
        else:
            holding.quantity = new_qty
            holding.total_invested = _round_money(new_invested)
            # avg_cost unchanged (keep precision normalized)
            holding.avg_cost = _round_price(avg_cost)

    else:
        raise HTTPException(status_code=400, detail="Invalid transaction type")

    # Insert immutable transaction record
    tx = Transaction(
        user_id=user_id,
        type=type,
        symbol=symbol,
        company_name=company_name,
        quantity=q,
        price=price,
        timestamp=timestamp,
    )
    db.add(tx)

    # Flush so tx.id is populated
    db.flush()

    return tx, holding

def update_market_price(db: Session, *, user_id: int, symbol: str, market_price: Decim
    ensure_user_exists(db, user_id)

    symbol = symbol.upper().strip()
    market_price = _round_price(_d(market_price))

    holding = get_holding_for_update(db, user_id, symbol)
    if holding is None:
        raise HTTPException(status_code=404, detail="Holding not found")

    holding.market_price = market_price
    db.flush()
    return holding

def compute_summary(db: Session, *, user_id: int):
    ensure_user_exists(db, user_id)

    holdings = db.query(Holding).filter(Holding.user_id == user_id).all()

    total_invested = sum((_d(h.total_invested) for h in holdings), start=Decimal("0"))
    total_market_value = sum((_d(h.market_price) * _d(h.quantity) for h in holdings), 

    total_invested = _round_money(total_invested)
```

```
    total_market_value = _round_money(total_market_value)

    unreal = _round_money(total_market_value - total_invested)

    if total_invested == 0:
        pct = None
    else:
        pct = (unreal / total_invested * Decimal("100")).quantize(Decimal("0.01"), rou

    return total_invested, total_market_value, unreal, pct
```

4.4 Routers (API Endpoints)

Create `app/api/routers/transactions.py`

Python

```python
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.api.deps import get_db
from app.schemas.transaction import TransactionCreate, TransactionOut
from app.schemas.holding import HoldingOut
from app.services.portfolio_service import apply_transaction

router = APIRouter(prefix="/transactions", tags=["transactions"])

@router.post("", response_model=dict)
def create_transaction(payload: TransactionCreate, db: Session = Depends(get_db)):
    with db.begin():  # atomic transaction
        tx, holding = apply_transaction(
            db,
            user_id=payload.user_id,
            type=payload.type,  # TransactionType (schema enum matches model enum valu
            symbol=payload.symbol,
            company_name=payload.company_name,
            quantity=payload.quantity,
            price=payload.price,
            timestamp=payload.timestamp,
        )

    market_value = holding.quantity * holding.market_price
    return {
        "transaction": TransactionOut.model_validate(tx),
        "holding": {
            **HoldingOut.model_validate(holding).model_dump(),
            "market_value": market_value,
        },
    }
```

Create `app/api/routers/holdings.py`

Python

```python
from decimal import Decimal
from fastapi import APIRouter, Depends, Query
from sqlalchemy.orm import Session

from app.api.deps import get_db
from app.models.holding import Holding
from app.schemas.holding import HoldingOut, MarketPriceUpdate
from app.services.portfolio_service import update_market_price

router = APIRouter(prefix="/holdings", tags=["holdings"])

@router.get("", response_model=list[HoldingOut])
def list_holdings(user_id: int = Query(..., ge=1), db: Session = Depends(get_db)):
    holdings = (
        db.query(Holding)
        .filter(Holding.user_id == user_id)
        .order_by(Holding.symbol.asc())
        .all()
    )
    # HoldingOut includes market_value, but it's computed; we'll inject it in FastAPI
    # easiest: return dicts; but keep it simple: HoldingOut has market_value field, so

    out: list[HoldingOut] = []
    for h in holdings:
        mv = Decimal(h.market_price) * Decimal(h.quantity)
        d = HoldingOut.model_validate(h).model_dump()
        d["market_value"] = mv
        out.append(HoldingOut(**d))
    return out

@router.patch("/{symbol}/market-price", response_model=HoldingOut)
def set_market_price(
    symbol: str,
    payload: MarketPriceUpdate,
    user_id: int = Query(..., ge=1),
    db: Session = Depends(get_db),
):
    with db.begin():
        holding = update_market_price(db, user_id=user_id, symbol=symbol, market_price=

    mv = Decimal(holding.market_price) * Decimal(holding.quantity)
    d = HoldingOut.model_validate(holding).model_dump()
    d["market_value"] = mv
    return HoldingOut(**d)
```

Create `app/api/routers/summary.py`

Python

```python
from fastapi import APIRouter, Depends, Query
from sqlalchemy.orm import Session

from app.api.deps import get_db
from app.schemas.summary import PortfolioSummary
from app.services.portfolio_service import compute_summary

router = APIRouter(prefix="/summary", tags=["summary"])

@router.get("", response_model=PortfolioSummary)
def get_summary(user_id: int = Query(..., ge=1), db: Session = Depends(get_db)):
    total_invested, total_market_value, unreal, pct = compute_summary(db, user_id=user_
    return PortfolioSummary(
        user_id=user_id,
        total_invested=total_invested,
        total_market_value=total_market_value,
        unrealized_gain_loss=unreal,
        gain_loss_pct=pct,
    )
```

4.5 Wire routers into `app/main.py`

Update `app/main.py` :

```python
from fastapi import FastAPI
from app.core.config import settings

from app.api.routers.transactions import router as transactions_router
from app.api.routers.holdings import router as holdings_router
from app.api.routers.summary import router as summary_router

app = FastAPI(title=settings.APP_NAME)

app.include_router(transactions_router)
app.include_router(holdings_router)
app.include_router(summary_router)

@app.get("/health")
def health():
    return {"status": "ok", "env": settings.ENV}
```

Run:

```bash
uvicorn app.main:app --reload
```

4.6 Data population (seed users)

Since no auth, you still need `users` rows so `user_id` is valid.

Option A: Simple seed script (recommended)

Create `scripts/seed.py` :

```bash
mkdir -p scripts
```

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from app.core.config import settings
from app.models.user import User

engine = create_engine(settings.DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

def main():
    db = SessionLocal()
    try:
        # idempotent-ish: only insert if empty
        if db.query(User).count() == 0:
            db.add_all([User(name="Analyst 1"), User(name="Analyst 2")])
            db.commit()
            print("Seeded users: Analyst 1, Analyst 2")
        else:
            print("Users already exist; skipping.")
    finally:
        db.close()

if __name__ == "__main__":
    main()
```

Run:

```bash
python scripts/seed.py
```

4.7 Quick manual test flow (via Swagger / curl)

1) Buy AAPL

```bash
curl -X POST http://127.0.0.1:8000/transactions \
  -H "Content-Type: application/json" \
  -d '{
    "user_id": 1,
    "type": "BUY",
    "symbol": "AAPL",
    "company_name": "Apple Inc",
    "quantity": 10,
    "price": 180.50
  }'
```

2) Set market price

```bash
curl -X PATCH "http://127.0.0.1:8000/holdings/AAPL/market-price?user_id=1" \
  -H "Content-Type: application/json" \
  -d '{"market_price": 190.00}'
```

3) Summary

```bash
curl "http://127.0.0.1:8000/summary?user_id=1"
```

4) Sell (valid)

```bash
curl -X POST http://127.0.0.1:8000/transactions \
  -H "Content-Type: application/json" \
  -d '{
    "user_id": 1,
    "type": "SELL",
    "symbol": "AAPL",
    "quantity": 4,
    "price": 191.00
  }'
```

5) Sell too much (should fail with 400)

Try selling 100.

Notes interviewers like

- Holdings row is locked with `FOR UPDATE` to prevent concurrent sells from going negative.
- Transactions are immutable audit log.
- Holdings is a derived/cached table for fast reads & portfolio summary

If you want, next step (Step 5) can be:

- Add `GET /transactions?user_id=&symbol=` (history) explain me what does schemas do here ??
- Add "rebuild holdings from transactions" admin endpoint (great safety valve)
- Add pagination + basic input normalization rules

Excellent question — this is one of the most important FastAPI concepts to understand clearly.

You now have:

- **models/** → SQLAlchemy ORM (database layer)
- **schemas/** → Pydantic models (API layer)

Let's break this down properly.

What Are Schemas?

In FastAPI:

> **Schemas are Pydantic models that define the structure of API input and output.**

They are NOT database models.

They are NOT tables.

They define:

- What the API accepts
- What the API returns

- Validation rules
- Serialization rules

Big Picture: 3 Layers

Think of your system like this:

Client → API → Service → Database

🔷 Schemas

Used between **Client ↔ API**

🔷 Models

Used between **API ↔ Database**

Example From Your Project

You created:

Code

```
app/schemas/transaction.py
```

Example:

Python

```python
class TransactionCreate(BaseModel):
    user_id: int
    type: TransactionType
    symbol: str
    quantity: int
    price: Decimal
```

This means:

👉 When someone sends POST /transactions
👉 FastAPI validates the request body using this schema
👉 If validation fails → automatic 422 error

What Happens Without Schemas?

If you didn't use schemas:

Python

```python
@app.post("/transactions")
def create_transaction(payload: dict):
```

Problems:

- No validation
- No automatic error responses
- No documentation typing

- No Swagger structure
- Risk of invalid data hitting business logic

Schemas solve ALL of that.

What Schemas Do For You

1️⃣ Validation

If someone sends:

```JSON
{
  "user_id": 1,
  "type": "BUY",
  "symbol": "AAPL",
  "quantity": -5,
  "price": 180
}
```

Because you defined:

```Python
quantity: int = Field(..., gt=0)
```

FastAPI automatically returns:

```Code
422 Unprocessable Entity
```

Without touching your service logic.

2️⃣ Automatic API Docs (Swagger)

Because of schemas:

- `/docs` shows correct request body structure
- It shows required fields
- It shows data types
- It shows example formats

This is HUGE for interviews.

3️⃣ Output Control (Very Important)

Your database model includes:

- internal fields
- maybe future internal fields
- timestamps
- internal IDs

But you don't always want to expose everything.

Example:

```python
class TransactionOut(BaseModel):
    id: int
    user_id: int
    type: TransactionType
    symbol: str
```

Only these fields get returned.

You control the API surface.

Why Not Just Use ORM Models Directly?

Because ORM models:

- Include database-only fields
- Include relationships
- May include internal metadata
- Are not designed for input validation

Also:

- You don't want your API tightly coupled to your DB schema.
- If DB changes, API shouldn't automatically break.

Clean Architecture Principle

You're following separation of concerns:

Layer	Responsibility
models	Database structure
schemas	API contracts
services	Business logic
routers	HTTP handling

Interviewers LOVE this structure.

Real Example Flow (Step-by-step)

User sends:

```json
{
  "user_id": 1,
  "type": "BUY",
  "symbol": "AAPL",
  "quantity": 10,
  "price": 180.5
}
```

Step 1

FastAPI parses request → validates against `TransactionCreate`

Step 2

You call:

```python
apply_transaction(...)
```

Step 3

Service updates ORM models (`Holding` , `Transaction`)

Step 4

You return:

```python
TransactionOut.model_validate(tx)
```

Schema converts ORM → JSON safely.

Why `from_attributes = True` ?

In your schema:

```python
class Config:
    from_attributes = True
```

This allows:

```python
TransactionOut.model_validate(tx)
```

Where `tx` is a SQLAlchemy model instance.

Pydantic reads attributes from ORM object.

Without this → it would fail.

Summary (The Clear Interview Answer)

If they ask:

> Why are you using schemas?

You say:

> "Schemas define the API contract. They validate request data, control response structure, and decouple the API layer from the database models. This ensures clean separation of concerns and prevents invalid data from reaching business logic."

That's a strong backend answer.

Quick Mental Model

Think of it like this:

Code

```
Client JSON  →  Schema  →  Service  →  ORM Model  →  Database
Database ORM →  Schema  →  JSON Response
```

Schemas = Gatekeepers of your API.

If you want next-level understanding, I can explain:

- Why we use separate CreateSchema / UpdateSchema / ResponseSchema
- Why Decimal handling matters for money
- Why you should never return ORM models directly
- Or how to simplify this for hackathon speed

Just tell me which direction you want to go.

I get this error when I run uvicorn app.main:app --reload

```
Traceback (most recent call last):
  File "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/process.py", line 313, in _bootstrap
    self.run()
    ~~~~~~~~~^^
  File "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/process.py", line 108, in run
    self._target(*self._args, **self._kwargs)
    ~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Users/jeevasaravanabhavanandam/Documents/Deloitte/Portfolio Holdings/.venv/lib/python3.13/site-packages/uvicorn/_subprocess.py", line 80, in subprocess_started
    target(sockets=sockets)
    ~~~~~~^^^^^^^^^^^^^^^^^^
```

```
  File "/Users/jeevasaravanabhavanandam/Documents/Deloitte/Portfolio Holdings/.venv/lib/python3.13/site-packages/uvicorn/server.py", line 75, in run
    return asyncio_run(self.serve(sockets=sockets), loop_factory=self.config.get_loop_factory())
  File "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/asyncio/runners.py", line 195, in run
    return runner.run(main)
           ~~~~~~~~~~^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/asyncio/runners.py", line 118, in run
    return self._loop.run_until_complete(task)
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "uvloop/loop.pyx", line 1518, in uvloop.loop.Loop.run_until_complete
  File "/Users/jeevasaravanabhavanandam/Documents/Deloitte/Portfolio Holdings/.venv/lib/python3.13/site-packages/uvicorn/server.py", line 79, in serve
    await self._serve(sockets)
  File "/Users/jeevasaravanabhavanandam/Documents/Deloitte/Portfolio Holdings/.venv/lib/python3.13/site-packages/uvicorn/server.py", line 86, in _serve
    config.load()
    ~~~~~~~~~~~^^
  File "/Users/jeevasaravanabhavanandam/Documents/Deloitte/Portfolio Holdings/.venv/lib/python3.13/site-packages/uvicorn/config.py", line 441, in load
    self.loaded_app = import_from_string(self.app)
                      ~~~~~~~~~~~~~~~~~~~^^^^^^^^^^
  File "/Users/jeevasaravanabhavanandam/Documents/Deloitte/Portfolio Holdings/.venv/lib/python3.13/site-packages/uvicorn/importer.py", line 19, in import_from_string
    module = importlib.import_module(module_str)
  File "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/importlib/__init__.py", line 88, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<frozen importlib._bootstrap>", line 1387, in _gcd_import
  File "<frozen importlib._bootstrap>", line 1360, in _find_and_load
  File "<frozen importlib._bootstrap>", line 1331, in _find_and_load_unlocked
  File "<frozen importlib._bootstrap>", line 935, in _load_unlocked
  File "<frozen importlib._bootstrap_external>", line 1026, in exec_module
  File "<frozen importlib._bootstrap>", line 488, in _call_with_frames_removed
  File
```

"/Users/jeevasaravanabhavanandam/Documents/Deloitte/Portfolio Holdings/app/main.py", line 4, in <module>
  from app.api.routers.transactions import router as transactions_router
  File "/Users/jeevasaravanabhavanandam/Documents/Deloitte/Portfolio Holdings/app/api/routers/transactions.py", line 7, in <module>
  from app.services.portfolio_service import apply_transaction
  File "/Users/jeevasaravanabhavanandam/Documents/Deloitte/Portfolio Holdings/app/services/portfolio_service.py", line 9, in <module>
  from app.models.holding import Holding
  File "/Users/jeevasaravanabhavanandam/Documents/Deloitte/Portfolio Holdings/app/models/holding.py", line 3, in <module>
  from app.db.base import Base
  File "/Users/jeevasaravanabhavanandam/Documents/Deloitte/Portfolio Holdings/app/db/base.py", line 9, in <module>
  from app.models.holding import Holding # noqa: E402,F401
  ^^

how to fix this
ImportError: cannot import name 'Holding' from partially initialized module 'app.models.holding' (most likely due to a circular import) (/Users/jeevasaravanabhavanandam/Documents/Deloitte/Portfolio Holdings/app/models/holding.py)

You hit a classic **circular import**:

- `app/models/holding.py` imports `Base` from `app/db/base.py`
- but `app/db/base.py` is importing `Holding`
- so Python ends up loading `holding.py` while `base.py` is still "half-built" → "partially initialized module"

Fix (recommended): remove model imports from `app/db/base.py`

✅ 1) Change `app/db/base.py` to ONLY define Base

```python
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass
```

No imports of models here. Period.

✅ 2) Create a separate file to import models for Alembic

Create: `app/db/base_imports.py`

```python
```

```python
# Import all models here so Alembic can discover them.
from app.models.user import User  # noqa: F401
from app.models.holding import Holding  # noqa: F401
from app.models.transaction import Transaction  # noqa: F401
```

This file is **only** for side-effect imports (metadata registration).

✅ 3) Update Alembic to load the models

In `alembic/env.py`, add an import of `base_imports` before `target_metadata`.

Find where you set `target_metadata` and do this:

```python
from app.db.base import Base
from app.db import base_imports  # noqa: F401  <-- add this line

target_metadata = Base.metadata
```

That's it. Alembic will now "see" your models without circular imports.

✅ 4) Restart uvicorn

```bash
uvicorn app.main:app --reload
```

Why this works

- Your ORM models can safely do: `from app.db.base import Base`
- `Base` no longer tries to import models back
- Alembic still discovers tables because `env.py` imports `base_imports` (which imports models)

Optional improvement (also helpful)

In your model files, make sure relationships use **string names** (you already did), e.g.

```python
user = relationship("User")
```

That avoids extra imports.

---

give me angular frontend for this work

Perfect 🔥 let's build a **clean Angular frontend** for your FastAPI Portfolio PMS.

We'll keep it:

- Simple
- Professional
- Hackathon-ready
- Easy to demo
- Clean architecture

We'll build:

1. Portfolio Dashboard
2. Add Transaction Form
3. Holdings Table
4. Portfolio Summary Card
5. Market Price Update

🚀 Step 1 — Create Angular Project

If Angular CLI is not installed:

```bash
npm install -g @angular/cli
```

Create project:

```bash
ng new portfolio-frontend
```

Choose:

- ✔️ Routing → Yes
- ✔️ Styles → CSS (or SCSS if you prefer)

Move into folder:

```bash
cd portfolio-frontend
```

Run it:

```bash
ng serve
```

Open:

```code
http://localhost:4200
```

🧠 Step 2 — Architecture

We'll structure it like this:

Code

```
src/app/
  core/
    services/
      api.service.ts
      portfolio.service.ts

  features/
    dashboard/
      dashboard.component.ts
      dashboard.component.html

    transactions/
      transaction-form.component.ts

    holdings/
      holdings-table.component.ts

  models/
    transaction.model.ts
    holding.model.ts
    summary.model.ts
```

📦 Step 3 — Create Models

`models/transaction.model.ts`

TypeScript

```typescript
export interface Transaction {
  id?: number;
  user_id: number;
  type: 'BUY' | 'SELL';
  symbol: string;
  company_name?: string;
  quantity: number;
  price: number;
  timestamp?: string;
}
```

`models/holding.model.ts`

TypeScript

```typescript
export interface Holding {
  user_id: number;
  symbol: string;
  company_name?: string;
  quantity: number;
  avg_cost: number;
  total_invested: number;
  market_price: number;
  market_value: number;
}
```

---

models/summary.model.ts

TypeScript

```typescript
export interface PortfolioSummary {
  user_id: number;
  total_invested: number;
  total_market_value: number;
  unrealized_gain_loss: number;
  gain_loss_pct?: number;
}
```

🌐 Step 4 — Create API Service

Generate:

Bash

```bash
ng generate service core/services/api
```

api.service.ts

TypeScript

```typescript
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class ApiService {

  private baseUrl = 'http://127.0.0.1:8000';

  constructor(private http: HttpClient) {}

  get<T>(url: string) {
    return this.http.get<T>(`${this.baseUrl}${url}`);
  }

  post<T>(url: string, body: any) {
    return this.http.post<T>(`${this.baseUrl}${url}`, body);
  }

  patch<T>(url: string, body: any) {
    return this.http.patch<T>(`${this.baseUrl}${url}`, body);
  }
}
```

📊 Step 5 — Portfolio Service

Bash

```bash
ng generate service core/services/portfolio
```

portfolio.service.ts

TypeScript

```
import { Injectable } from '@angular/core';
import { ApiService } from './api.service';
import { Transaction } from '../../models/transaction.model';
import { Holding } from '../../models/holding.model';
import { PortfolioSummary } from '../../models/summary.model';

@Injectable({
  providedIn: 'root'
})
export class PortfolioService {

  private userId = 1;

  constructor(private api: ApiService) {}

  getHoldings() {
    return this.api.get<Holding[]>(`/holdings?user_id=${this.userId}`);
  }

  getSummary() {
    return this.api.get<PortfolioSummary>(`/summary?user_id=${this.userId}`);
  }

  createTransaction(tx: Transaction) {
    return this.api.post(`/transactions`, tx);
  }

  updateMarketPrice(symbol: string, price: number) {
    return this.api.patch(
      `/holdings/${symbol}/market-price?user_id=${this.userId}`,
      { market_price: price }
    );
  }
}
```

🖥️ Step 6 — Dashboard Component

Generate:

```bash
ng generate component features/dashboard
```

`dashboard.component.ts`

```typescript
```

```
import { Component, OnInit } from '@angular/core';
import { PortfolioService } from '../../core/services/portfolio.service';
import { Holding } from '../../models/holding.model';
import { PortfolioSummary } from '../../models/summary.model';

@Component({
  selector: 'app-dashboard',
  templateUrl: './dashboard.component.html'
})
export class DashboardComponent implements OnInit {

  holdings: Holding[] = [];
  summary?: PortfolioSummary;

  constructor(private portfolioService: PortfolioService) {}

  ngOnInit() {
    this.loadData();
  }

  loadData() {
    this.portfolioService.getHoldings()
      .subscribe(data => this.holdings = data);

    this.portfolioService.getSummary()
      .subscribe(data => this.summary = data);
  }
}
```

---

dashboard.component.html

HTML

```
<h2>Portfolio Summary</h2>

<div *ngIf="summary">
  <p>Total Invested: {{ summary.total_invested | number:'1.2-2' }}</p>
  <p>Total Market Value: {{ summary.total_market_value | number:'1.2-2' }}</p>
  <p>Unrealized Gain/Loss:
    <span [ngClass]="{
      'positive': summary.unrealized_gain_loss >= 0,
      'negative': summary.unrealized_gain_loss < 0
    }">
      {{ summary.unrealized_gain_loss | number:'1.2-2' }}
    </span>
  </p>
</div>

<hr>

<h2>Holdings</h2>

<table border="1">
  <tr>
    <th>Symbol</th>
    <th>Quantity</th>
    <th>Avg Cost</th>
    <th>Market Price</th>
    <th>Market Value</th>
  </tr>

  <tr *ngFor="let h of holdings">
    <td>{{ h.symbol }}</td>
    <td>{{ h.quantity }}</td>
    <td>{{ h.avg_cost | number:'1.2-2' }}</td>
    <td>{{ h.market_price | number:'1.2-2' }}</td>
    <td>{{ h.market_value | number:'1.2-2' }}</td>
  </tr>
</table>
```

🔧 Step 7 — Enable HTTP Module

Open `app.module.ts`

Add:

TypeScript

```typescript
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  declarations: [...],
  imports: [
    BrowserModule,
    AppRoutingModule,
    HttpClientModule
  ],
})
```

🛑 Important — CORS Fix

In FastAPI, add this in `main.py` :

Python

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:4200"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Restart backend.

🎯 What You Now Have

✔ Angular dashboard
✔ Holdings table
✔ Portfolio summary

⭐ If You Want Next

I can now help you:

- Add Transaction Form UI
- Add Market Price inline edit
- Add Profit/Loss color styling
- Add Bootstrap/Material styling
- Make it look Deloitte-level polished
- Add charts (Chart.js)
- Add transaction history screen

Tell me how polished you want this to be for Hack2Hire and I'll level it up accordingly 🚀

✅ Recommended Structure (Professional Setup)

Create a root folder:

Code

```
portfolio-pms/
│
├── backend/
│   ├── app/
│   ├── alembic/
│   ├── .env
│   └── ...
│
├── frontend/
│   ├── src/
│   ├── angular.json
│   └── ...
│
└── README.md
```

Why Separate Them?

should the frontend be in the same folder as backend portfolio-pms-backend or different folder ?

Excellent question — this is an **architecture decision**, and interviewers absolutely care about it.

Short answer:

👉 **Frontend and backend should be in separate folders**
👉 But inside the same parent project directory

1️⃣ Clean Separation of Concerns

Backend:

- API
- Database
- Business logic

Frontend:

- UI
- State
- HTTP calls

They are two different applications.

2️⃣ Deployment Reality

In real life:

- Backend deploys to:
  - Docker container
  - AWS ECS
  - Azure App Service
- Frontend deploys to:
  - S3
  - Vercel
  - Netlify
  - CDN

They are built and deployed independently.

Keeping them separate makes that easy.

3️⃣ Tech Independence

You could later replace:

- Angular → React
- FastAPI → Node

Without touching the other.

❌ What Not To Do

Don't put Angular inside:

```
portfolio-pms-backend/app/frontend
```

That's messy and tightly coupled.

💡 Hackathon Tip

If you already created:

```
Code
portfolio-pms-backend/
```

Just move up one level and create:

```bash
cd ..
mkdir portfolio-pms
mv portfolio-pms-backend portfolio-pms/backend
cd portfolio-pms
ng new frontend
```

Now you're clean.

🧠 Interview-Ready Answer

If they ask:

> "How did you structure the project?"

You say:

> "I structured it as a mono-repo with separate backend and frontend directories to maintain clean separation of concerns and allow independent builds and deployments."

That sounds senior.

When Would You Combine Them?

Only if:

- Backend serves static Angular build files
- Small internal app
- Very simple deployment

But even then — still separate during development.

Final Recommendation For You

Since this is Deloitte Hack2Hire:

✔ Put frontend in separate folder
✔ Keep root-level README explaining architecture
✔ Run backend on 8000
✔ Run Angular on 4200

Very clean demo.

If you want next:

I can show you:

- How to configure environment files properly (dev/prod)
- How to build Angular and serve it from FastAPI (optional)
- Or how to Dockerize both properly

where should be the venv

Very good question — this is exactly the kind of detail that shows whether someone understands real project structure.

✅ Correct Answer

The **venv should live inside the backend folder**, not at the root.

✅ Recommended Structure

Plain text

```
portfolio-pms/
│
├── backend/
│   ├── .venv/              ← HERE
│   ├── app/
│   ├── alembic/
│   ├── requirements.txt
│   └── .env
│
├── frontend/
│   ├── node_modules/
│   ├── src/
│   └── package.json
│
└── README.md
```

Why Inside `backend/` ?

Because:

- The venv is **Python-specific**
- Only the backend uses Python
- The frontend uses Node.js (npm/yarn)

They have completely different runtimes.

❌ What NOT To Do

Do NOT put `.venv` at the root:

Plain text

```
portfolio-pms/
│
├── .venv/    ❌
├── backend/
├── frontend/
```

That mixes environments and becomes messy.

Even Better (Clean Setup Steps)

If you're restructuring:

```bash
mkdir portfolio-pms
mv portfolio-pms-backend portfolio-pms/backend
cd portfolio-pms/backend
python -m venv .venv
source .venv/bin/activate
```

Then reinstall dependencies:

```bash
pip install -r requirements.txt
```

Also Add to `.gitignore`

Inside `backend/.gitignore`:

```plain text
.venv/
__pycache__/
*.pyc
.env
```

Inside `frontend/.gitignore` (Angular already has this):

```plain text
node_modules/
dist/
```

Interview-Level Explanation

If asked:

> "Where do you keep your virtual environment?"

You say:

> "Inside the backend directory, since it's Python-specific. The frontend has its own Node dependency system, so keeping environments separated avoids cross-contamination and keeps the repo clean."

That's a clean, professional answer.

If you'd like, next I can:

- Help you reorganize your current structure cleanly
- Or help you connect Angular properly to backend and test end-to-end

Your backend is solid — now we can polish the frontend demo.

? Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? (y/N)

For your **Hack2Hire FastAPI + Angular portfolio app**, the correct choice is:

👉 **N (No)**

Just press:

```
N
```

Why You Should Choose NO

You are building:

- A dashboard app
- Internal tool
- API-driven frontend
- Not a public SEO website

SSR/SSG is mainly for:

- SEO optimization
- Public marketing sites
- Performance-first content sites
- Blogs
- E-commerce

Your app is:

- Data-heavy
- Auth-based (eventually)
- Pure API consumption
- Not SEO-dependent

So SSR adds:

- More complexity
- Extra build configuration
- Unnecessary deployment complexity

What Happens If You Say Yes?

Angular will:

- Add Angular Universal
- Add server build config
- Add Node server
- Complicate your setup
- Increase cognitive load

For a 4.5-hour build → not worth it.

Interview Smart Answer

If someone asks:

> Why didn't you enable SSR?

You say:

> "Since this is an internal dashboard-style application consuming APIs and not SEO-driven, client-side rendering is sufficient and keeps the architecture simpler."

That's exactly the right reasoning.

So go ahead:

👉 Type **N**

Then we continue building the UI cleanly.

? Which AI tools do you want to configure with Angular best practices? https://angular.dev/ai/develop-with-ai
❯◉ None
◯ Agents.md [https://agents.md/]
◯ Claude [https://docs.anthropic.com/en/docs/claude-code/memory]
◯ Cursor [https://docs.cursor.com/en/context/rules]
◯ Gemini [https://ai.google.dev/gemini-api/docs]
◯ GitHub Copilot [https://code.visualstudio.com/docs/copilot/copilot-customization]
◯ JetBrains AI [https://www.jetbrains.com/help/junie/customize-guidelines.html]? Which AI tools do you want to configure with Angular best practices? https://angular.dev/ai/develop-with-ai
❯◉ None
◯ Agents.md [https://agents.md/]
◯ Claude [https://docs.anthropic.com/en/docs/claude-code/memory]
◯ Cursor [https://docs.cursor.com/en/context/rules]
◯ Gemini [https://ai.google.dev/gemini-api/docs]
◯ GitHub Copilot [

https://code.visualstudio.com/docs/copilot/copilot-customization]
◯ JetBrains AI [https://www.jetbrains.com/help/junie/customize-guidelines.html]

Choose ◉ **None**.

That prompt just generates optional "AI best practices" config files (like instructions/rules for specific IDE assistants). It won't affect your Angular app.

What to do right now

- Keep **None** selected
- Press **Enter**

When would you pick something else?

Only if you *know* you're using that tool heavily in this repo:

- **GitHub Copilot** → if you want repo-specific Copilot instructions
- **Cursor / JetBrains AI / Claude** → if you're using those editors/tools

For Hack2Hire speed + less noise in the repo: **None is best**.

I dont have app.module.ts

Good — this means you created a **standalone Angular app** (Angular 15+ default).

Angular no longer uses `AppModule` by default.

So instead of `app.module.ts`, you have:

Code

```
src/main.ts
src/app/app.config.ts
src/app/app.component.ts
```

This is correct and modern.

✅ What Changed in Standalone Angular?

Old (NgModule-based):

Code

```
app.module.ts
```

New (Standalone-based):

Code

```
main.ts
app.config.ts
```

Angular moved away from modules.

🚀 How To Add HttpClient (Standalone Way)

Since there is no `app.module.ts`, you must register providers in:

`src/app/app.config.ts`

Open it and modify like this:

TypeScript

```typescript
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient } from '@angular/common/http';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient()   // 👈 ADD THIS
  ]
};
```

That replaces:

TypeScript

```typescript
HttpClientModule
```

from old Angular.

🧠 Why This Works

Standalone Angular uses:

TypeScript

```typescript
provideHttpClient()
```

instead of:

TypeScript

```typescript
HttpClientModule
```

Much cleaner.

🔍 How To Know You're Using Standalone?

Check `main.ts`. If you see:

TypeScript

```typescript
bootstrapApplication(AppComponent, appConfig)
```

You are using standalone architecture.

✅ What You Should Do Now

1. Add `provideHttpClient()` in `app.config.ts`
2. Save
3. Restart Angular:

```bash
ng serve
```

⚡ Quick Reality Check

Standalone Angular is:

- More modern
- Less boilerplate
- Cleaner
- Better for hackathons

You're actually in a better setup.

If you want, next I can:

- Show how to properly structure standalone components
- Wire routing cleanly
- Or polish the dashboard UI to look Deloitte-ready

Tell me what you want next.